Building a pet-sitting marketplace as an AI development experiment

Over the past few weeks I've been building MySitter — a Rover-style pet and house sitting marketplace — mostly to see how far I can push AI-assisted development on a real, non-trivial product.

The stack is what you'd expect for this kind of app: FastAPI backend, React + Vite for web, Expo (React Native) for mobile, PostgreSQL with PostGIS for geographic search, Redis + Celery for background jobs, Stripe for subscriptions, WebSockets for real-time messaging. Passkey auth because passwords are annoying.

The interesting part is the workflow. Gemini writes each phase of the implementation — I spec it out, it generates the code. Then Claude reviews the output before anything gets committed. Not "does this look okay" review — actual systematic review: does every hook have an `enabled` guard? Are shared types being used or are inline types getting redeclared? Is any `useQuery` being called outside its `QueryClientProvider`? I track recurring failure patterns as explicit rules and feed them back into the next Gemini prompt.

By Phase 8 we're at 20+ backend tests, rate limiting, HTML email templates, Stripe checkout, push notifications, referral rewards, and a fully functional mobile app. The AI writes most of the code. I write the architecture, catch the bugs, and decide what gets built next.

What I've learned: the bottleneck isn't generation speed, it's knowing what questions to ask. "Write me a booking system" produces garbage. "Write me a booking service that enforces PENDING → CONFIRMED → COMPLETED state transitions, rejects cancellations after completion, and fires a Celery notification task on each transition" produces something you can actually use.

Still a lot to learn about where these tools are reliable and where they're not. But it's a genuinely different way to build software.